|UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO
UNIDAD DE ESTUDIOS SUPERIORES JIQUIPILCO
Ingeniería en Sistemas Computacionales

ACCIONES SEMÁNTICAS Y TABLA DE SÍMBOLOS

DECLARAR ANTES DE USAR

TIPO DE TRABAJO

Practica

ASIGNATURA

Programación de Sistemas

PRESENTA

Diego Deciderio García

Matrícula 10240041

DOCENTE

Mtro. Eduardo Quirino Sanroman

GRUPO	SEMESTRE	FECHA
10SC151	Quinto	06/10/2026

Checkpoint 1: Verificar el punto de partida

Objetivo

Ubicar el proyecto, compilar el esqueleto y confirmar que el programa avisa que la tabla aún no está terminada.

```
Equipo1@TechNova-Group:~$ cd ~/compint/actividades
Equipo1@TechNova-Group:~/compint/actividades$ cd A06_semantica
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ls -R
.:
Makefile                        mini_parser.h:Zone.Identifier
Makefile:Zone.Identifier        pruebas
main.c                          tabla_simbolos.c
main.c:Zone.Identifier          tabla_simbolos.c:Zone.Identifier
mini_parser.c                   tabla_simbolos.h
mini_parser.c:Zone.Identifier   tabla_simbolos.h:Zone.Identifier
mini_parser.h

./pruebas:
redeclaracion.ci                        uso_antes.ci:Zone.Identifier
redeclaracion.ci:Zone.Identifier        uso_repetido.ci
sintaxis_invalida.ci                    uso_repetido.ci:Zone.Identifier
sintaxis_invalida.ci:Zone.Identifier    valido.ci
uso_antes.ci                            valido.ci:Zone.Identifier
```

Ilustración 1 Comandos ejecutados

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c main.c
gcc -std=c11 -Wall -Wextra -pedantic -c mini_parser.c
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolos
.o -o compint_a06
Base de Actividad 6 construida correctamente.
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a0
6 < pruebas/valido.ci
Actividad incompleta: falta conectar la tabla de simbolos.
--- Tabla de simbolos (0 entradas) ---

Analisis semantico exitoso.
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$
```

Ilustración 2 Construcción del entorno

Checkpoint 2: Representar los tipos

Objetivo

Definir los tipos de dato del lenguaje y la estructura de una entrada de la tabla de símbolos.

Qué se completó

1. **TODO 1** — enumeración TipoDato con T_ENTERO y T_REAL.

2. **TODO 2** — estructura Simbolo con nombre, tipo y línea de declaración.

```c
#ifndef TABLA_SIMBOLOS_H
#define TABLA_SIMBOLOS_H

#define MAX_SIMBOLOS 64

typedef enum {
    T_ENTERO,
    T_REAL
} TipoDato;

typedef struct {
    char nombre[64];
    TipoDato tipo;
    int linea_decl;
} Simbolo;

extern int errores_semanticos;

void ts_inicializa(void);
int  ts_busca(const char *nombre);
void ts_declara(const char *nombre, TipoDato tipo, int linea);
void ts_usa(const char *nombre, int linea);
void ts_imprime(void);

#endif
```

Ilustración 3 Código *tabla_simbolos.h*

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c main.c
gcc -std=c11 -Wall -Wextra -pedantic -c mini_parser.c
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolos
.o -o compint_a06
Base de Actividad 6 construida correctamente.
```

Ilustración 4 Comprobación

Qué se aprendió

- Una entrada (Simbolo) representa **una** variable, no la tabla completa.
- T_ENTERO / T_REAL corresponden a int / float de CompInt.
- linea_decl guarda dónde se declaró por primera vez.

Checkpoint 3: Preparar el almacenamiento

Objetivo

Reservar el arreglo de símbolos y el contador de entradas ocupadas.

```
4
5   static Simbolo tabla[MAX_SIMBOLOS];
6   static int cantidad = 0;
```

Ilustración 5 Líneas modificadas

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolos
.o -o compint_a06
Base de Actividad 6 construida correctamente.
```

Ilustración 6 Comprobación

Checkpoint 4: Buscar un nombre

Objetivo

Implementar ts_busca: devuelve el índice si el nombre existe, o -1 si no.

```
13  void reporta_error(int linea, const char *nombre, const char *msg)
14  {
15      fprintf(stderr, "Error semantico (linea %d): variable '%s' %s\n",
16              linea, nombre, msg);
17      errores = 1;
18  }
19
20  void ts_inicializa(void)
21  {
22      cantidad = 0;
23      errores = 0;
24  }
25
26  int ts_busca(const char *nombre)
27  {
28      int i;
29      for (i = 0; i < cantidad; i++) {
30          if (strcmp(tabla[i].nombre, nombre) == 0)
31              return i;
32      }
33      return -1;
34  }
```

Ilustración 7 Sección modificada

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c main.c
gcc -std=c11 -Wall -Wextra -pedantic -c mini_parser.c
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolos
.o -o compint_a06
Base de Actividad 6 construida correctamente.
```

Ilustración 8 Comprobación

Checkpoint 5. Registra una declaración

objetivo

En este paso completamos la función ts_declara, que tiene dos responsabilidades:

1. **TODO 5** → Detectar redeclaraciones: si el nombre ya existe en la tabla, reportar error y no hacer nada más.

2. **TODO 6** → Registrar la nueva entrada en el siguiente casillero libre.

```c
void ts_declara(const char *nombre, TipoDato tipo, int linea)
{
    /* Protección: tabla llena */
    if (cantidad >= MAX_SIMBOLOS) {
        reporta_error(linea, nombre, "tabla llena");
        return;
    }

    /* TODO 5: detectar redeclaración */
    if (ts_busca(nombre) >= 0) {
        reporta_error(linea, nombre, "ya declarada");
        return;
    }

    /* TODO 6: registrar la nueva entrada */
    strncpy(tabla[cantidad].nombre, nombre, 63);
    tabla[cantidad].nombre[63] = '\0';   /* asegurar terminador */
    tabla[cantidad].tipo       = tipo;
    tabla[cantidad].linea_decl = linea;
    cantidad++;
}
```

Ilustración 9 Bloque modificado

Detalles importantes

- **strncpy(..., 63) + nombre[63] = '\0'**: strncpy no garantiza el terminador nulo si el origen es más largo que el tamaño. Forzar el \0 en la posición 63 evita desbordamientos y lecturas basura.
- **return; tras reportar error**: no incrementamos cantidad ni escribimos en la tabla. Así la entrada original queda intacta.
- **ts_busca(nombre) >= 0**: recuerda que ts_busca devuelve el índice (≥ 0) si lo encuentra, o -1 si no.

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make clean
rm -f main.o mini_parser.o tabla_simbolos.o compint_a06
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c main.c
gcc -std=c11 -Wall -Wextra -pedantic -c mini_parser.c
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolos
.o -o compint_a06
Base de Actividad 6 construida correctamente.
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a0
6 < pruebas/redeclaracion.ci
--- Tabla de simbolos (0 entradas) ---

Analisis semantico exitoso.
```

Ilustración 10 Compilación y prueba

Checkpoint 6. Comprueba un uso

Objetivo

Completar ts_usa, que se invoca cada vez que el parser encuentra un identificador en una asignación o en print. Su única responsabilidad es **consultar** la tabla: si la variable no existe, reporta error.

```
57  void ts_usa(const char *nombre, int linea)
58  {
59      if (ts_busca(nombre) < 0)
60          reporta_error(linea, nombre, "no declarada");
61  }
```

Ilustración 11 Sección modificada

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make clean
rm -f main.o mini_parser.o tabla_simbolos.o compint_a06
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c main.c
gcc -std=c11 -Wall -Wextra -pedantic -c mini_parser.c
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolo
.o -o compint_a06
Base de Actividad 6 construida correctamente.
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a
6 < pruebas/uso_antes.ci
--- Tabla de simbolos (0 entradas) ---

Analisis semantico exitoso.
```

Ilustración 12 Compilación y prueba

Checkpoint 7. Conecta las acciones semánticas

Ahora toca enlazar el parser con la tabla de símbolos. Abre mini_parser.c y busca los comentarios TODO 8 y TODO 9.

Objetivo

- **TODO 8**: después de reconocer una declaración completa (int x; o float x;), llamar a ts_declara.
- **TODO 9** (aparece en dos lugares): después de reconocer un uso de identificador (en asignación y en print), llamar a ts_usa.

```c
static void declaracion(void)
{
    TipoDato tipo = ...;   /* ya está resuelto */
    char nombre[64];
    /* ... lexer lee el identificador en 'nombre' ... */
    /* ... consume el ';' ... */

    /* TODO 8 */
    ts_declara(nombre, tipo, linea_actual);
}
```

Ilustración 13 Bloque agregado y mas

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make clean
rm -f main.o mini_parser.o tabla_simbolos.o compint_a06
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ make
gcc -std=c11 -Wall -Wextra -pedantic -c main.c
gcc -std=c11 -Wall -Wextra -pedantic -c mini_parser.c
gcc -std=c11 -Wall -Wextra -pedantic -c tabla_simbolos.c
gcc -std=c11 -Wall -Wextra -pedantic main.o mini_parser.o tabla_simbolos.o -o compint_a06
Base de Actividad 6 construida correctamente.
```

Ilustración 14 Comprobación

Pruebas

./compint_a06 < pruebas/valido.ci

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a06 < pruebas/valido.ci          && echo "OK estado 0"
--- Tabla de simbolos (2 entradas) ---
[0] contador         int   declarada en linea 1
[1] promedio         float declarada en linea 2

Analisis semantico exitoso.
OK estado 0
```

./compint_a06 < pruebas/uso_antes.ci

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a06 < pruebas/uso_antes.ci
Error semantico (linea 1): variable 'x' no declarada
--- Tabla de simbolos (1 entradas) ---
[0] x               int   declarada en linea 2

Analisis finalizado con 0 error(es) sintactico(s) y 1 error(es) semantico(s).
```

./compint_a06 < pruebas/redeclaracion.ci

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a06 < pruebas/redeclaracion.ci
Error semantico (linea 2): variable 'total' ya declarada
--- Tabla de simbolos (1 entradas) ---
[0] total           int   declarada en linea 1

Analisis finalizado con 0 error(es) sintactico(s) y 1 error(es) semantico(s).
```

./compint_a06 < pruebas/uso_repetido.ci

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a06 < pruebas/uso_repetido.ci
--- Tabla de simbolos (1 entradas) ---
[0] dato            int   declarada en linea 1

Analisis semantico exitoso.
```

./compint_a06 < pruebas/sintaxis_invalida.ci

```
Equipo1@TechNova-Group:~/compint/actividades/A06_semantica$ ./compint_a06 < pruebas/sintaxis_invalida.ci
Error sintactico (linea 1): sentencia no reconocida
--- Tabla de simbolos (0 entradas) ---

Analisis finalizado con 1 error(es) sintactico(s) y 0 error(es) semantico(s).
```

Síntesis

Una sentencia puede estar sintácticamente correcta pero semánticamente incorrecta porque la sintaxis solo verifica la forma (orden de tokens), mientras la semántica verifica el significado. Por ejemplo, x = 5; tiene la estructura válida de una asignación, pero si x nunca fue declarada, la sentencia carece de sentido: no se sabe de qué tipo es ni dónde está almacenada. Por otro lado, una redeclaración no debe reemplazar la primera entrada porque en un ámbito global cada nombre tiene un único tipo asociado. Si se permitiera sobrescribir, los usos anteriores ya validados contra el tipo original quedarían inconsistentes, y el compilador no podría garantizar coherencia en todo el programa.